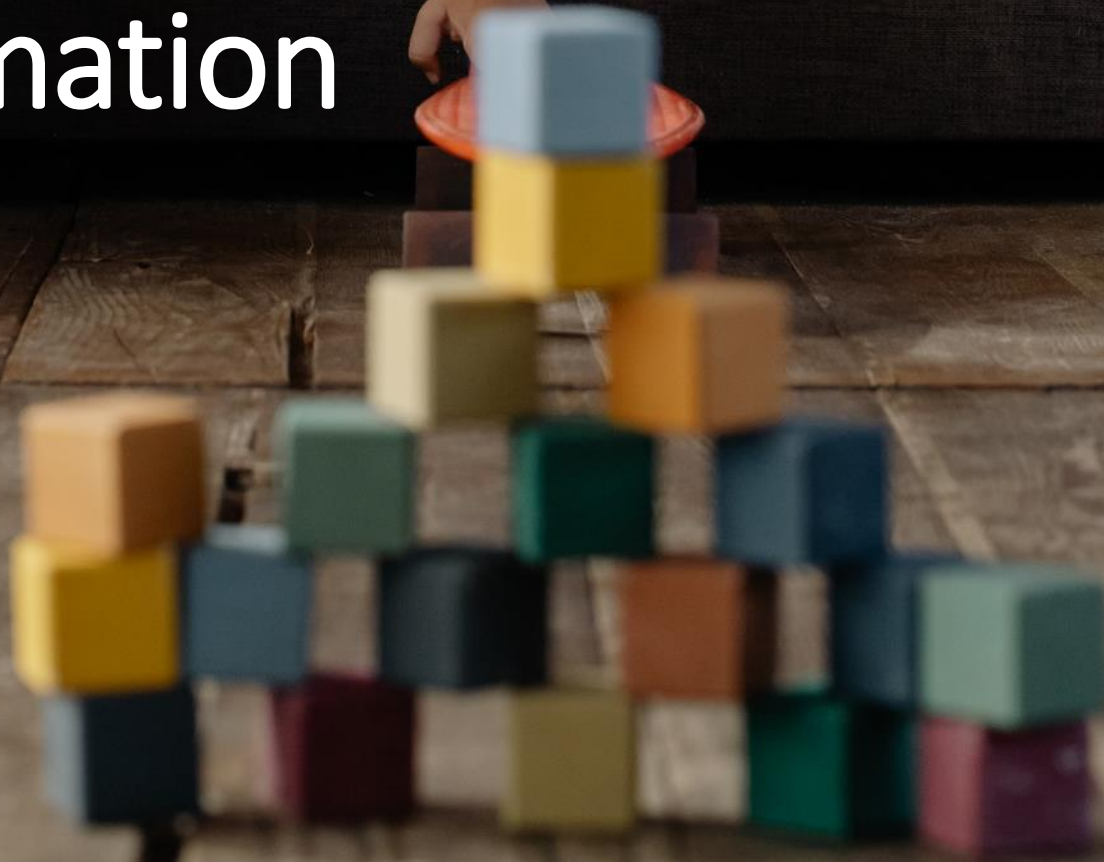


Programmation Orientée Objets



Origine

Programmation structurée ou procédurale limitée

Programme de + en + complexe

Systèmes d'exploitation graphiques



Approche « objet »

Décomposer un problème en un certain nombre d'entités indépendantes les unes des autres

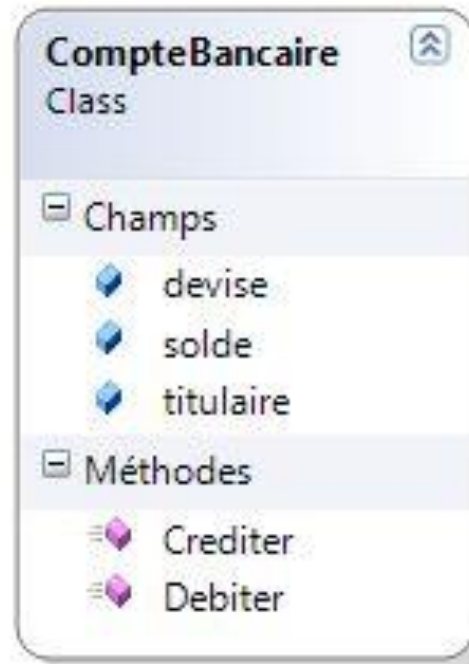
Résoudre un problème :

- Identification des constituants du système
- Analyse des relations qui existent entre eux

Exemple : modélisons...



Un compte bancaire



- En groupe, modélisez...



...une recette de cuisine !



Avantages

- Grande structuration du projet
 - Réflexion préalable avant le codage
 - Décomposition naturelle du système
- Indépendance des composants
 - Développement, tests et maintenance indépendants pour chaque composant
 - Intégration plus facile grâce aux interfaces
 - Documentation



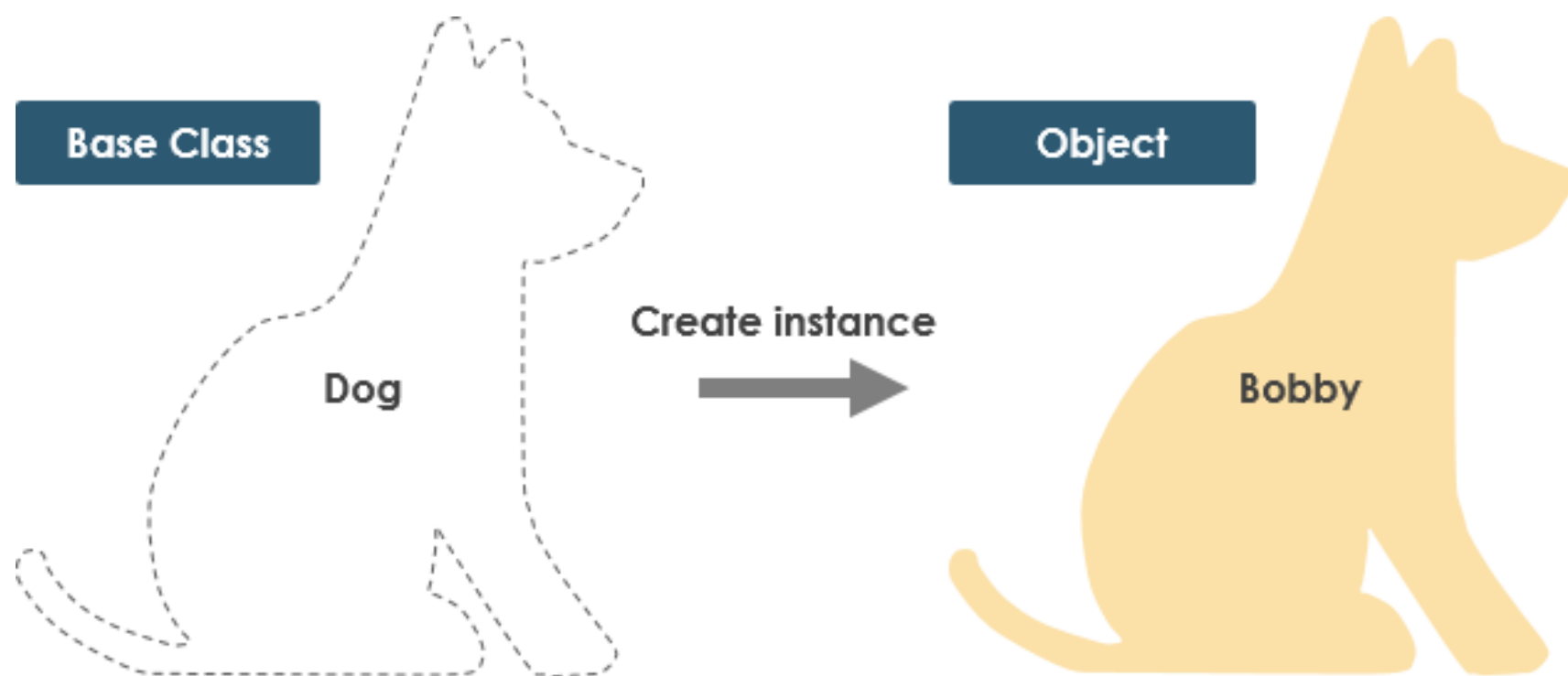
Inconvénients

- Conception plus longue
- Respecter les règles de l'art ou...
 - Système extrêmement complexe...
 - ... et difficile à faire évoluer
 - Composants inutiles
 - Mauvaises performances



Concepts de base

- **Classe** : un nouveau type de données que nous créons, composé de **membres** de 2 sortes :
 - **Attributs** : données qui décrivent l'état de la classe
 - **Méthodes** : fonctions pour agir sur les données
- **Objet** = instance de classe (les "variables" qui vont utiliser notre nouveau type)

**Properties**

Color

Eye Color

Height

Length

Weight

Methods

Sit

Lay Down

Shake

Come

Property Values

Color: Yellow

Eye Color: Brown

Height: 17 in

Length: 35 in

Weight: 24 pounds

Methods

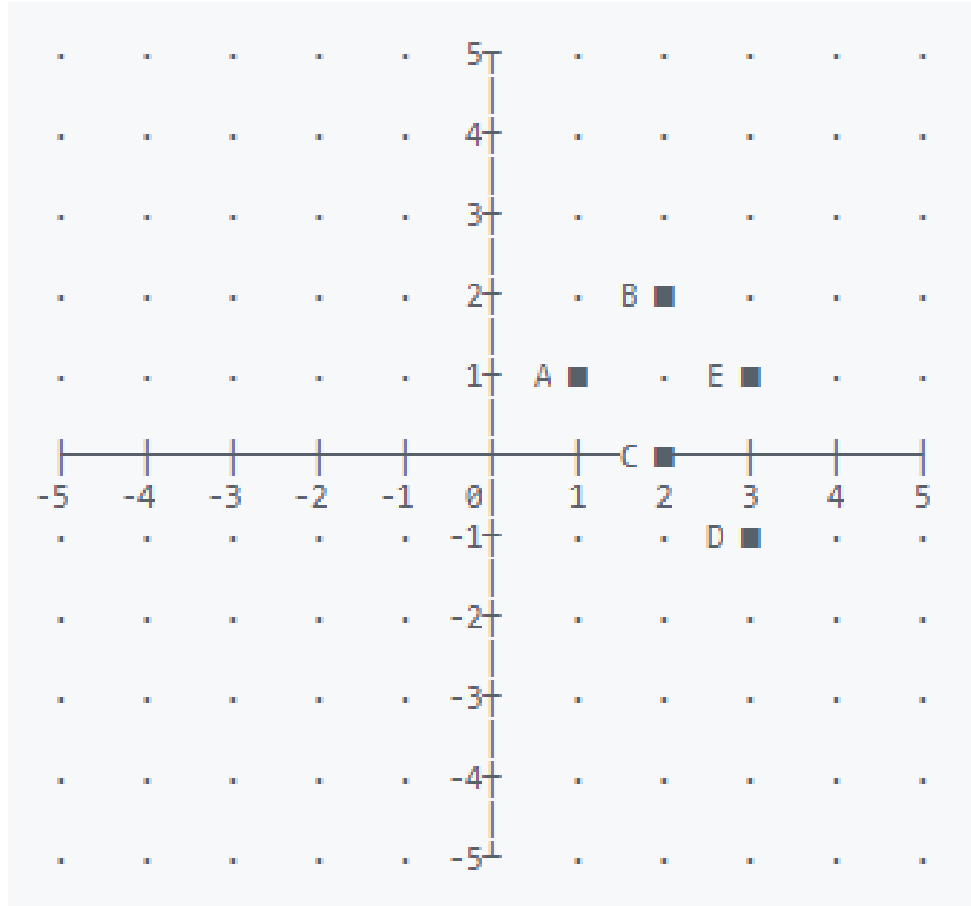
Sit

Lay Down

Shake

Come

Concepts de base - Exemple



- Pour réaliser un programme, permettant de manipuler des points dans un espace à deux dimensions :

- Classe :

Point
+ x : int
+ y : int
+ lettre : char
+ afficher()

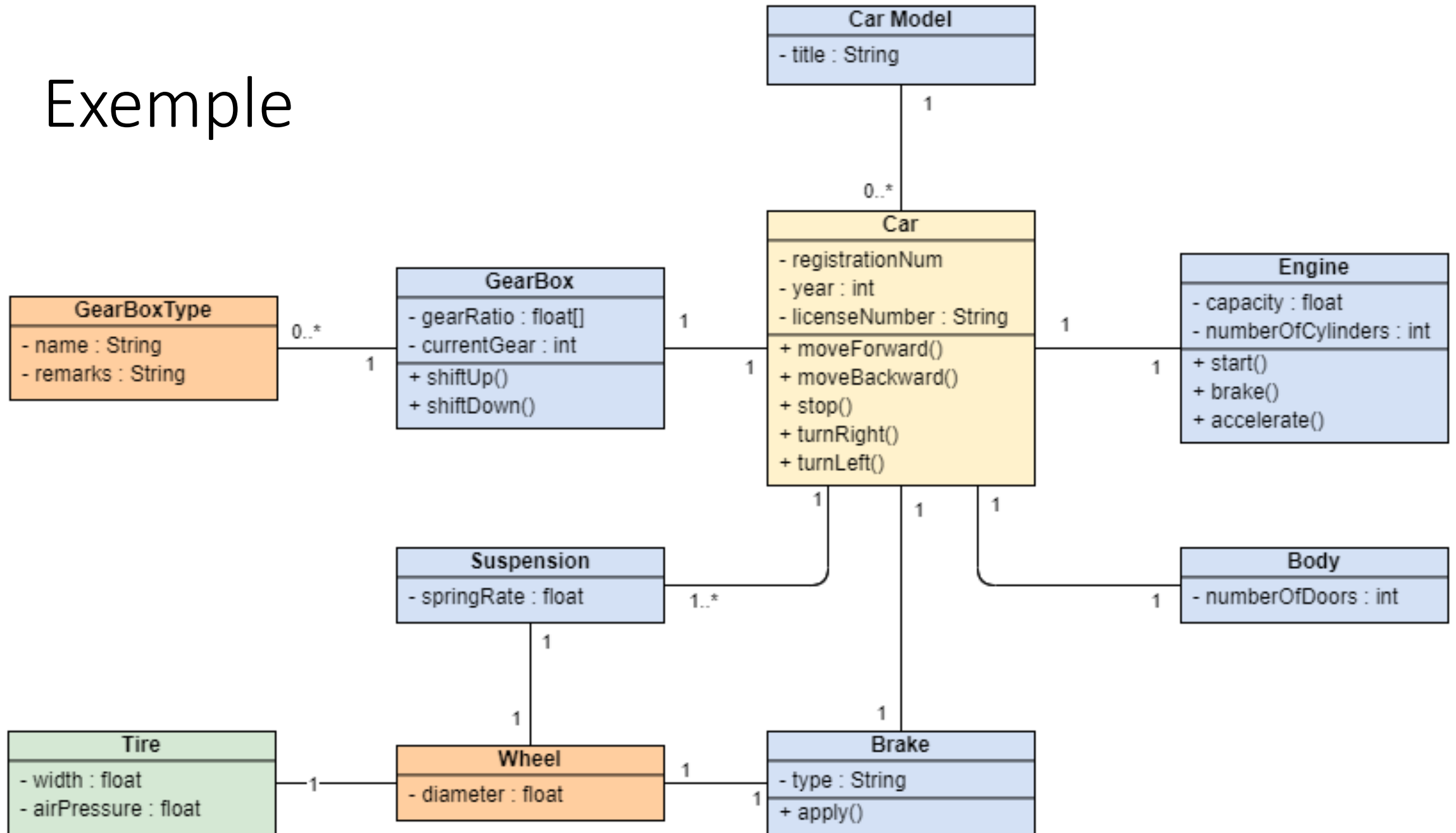
- Objets : A(1, 1), B(2, 2), C(2, 0)...

Diagramme de classes

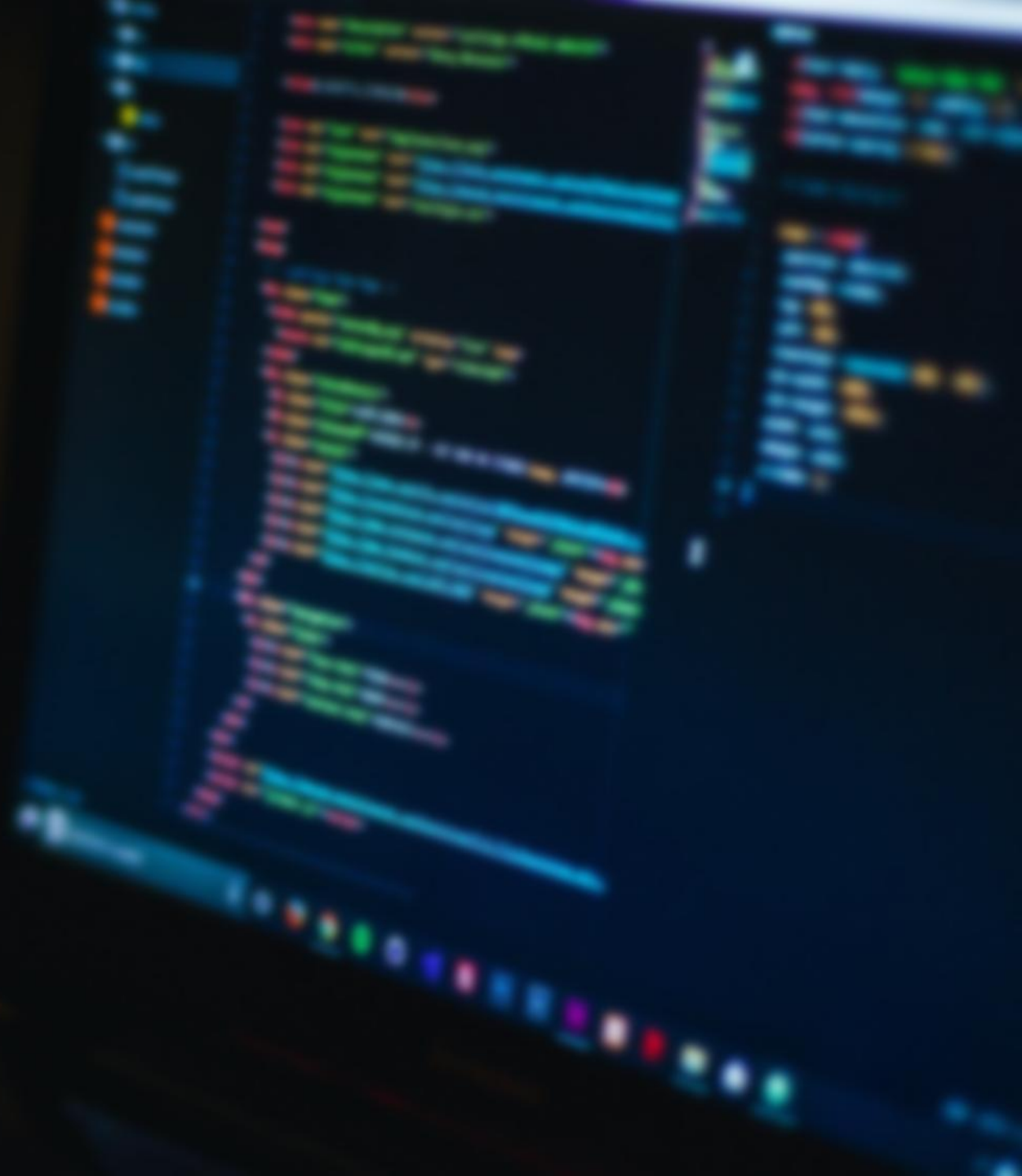
- Unified Modeling Language (UML)
- Schéma présentant les classes d'un programme et leurs relations

NomDeLaClasse
- attribut1 : type ... + attributN : type
+ methode1(typeParametre,...) : typeRetour ... - methodeN(typeParametre,...) : typeRetour

Example



Et en C++ ?



Déclaration

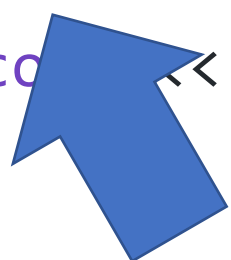
```
// Déclaration de la classe Point
class Point {      // Nom de la classe
    public :       // Déclaration des membres publics
        int x;
        int y;
        char lettre;
        void afficher();
};
```

Point
+ x : int
+ y : int
+ lettre : char
+ afficher()

Définition des méthodes #2

// Définition en-dehors de la classe

```
void Point::afficher() {  
    std::cout << lettre << "(" << x << "," << y << " )";  
}
```



Point
+ x : int
+ y : int
+ lettre : char
+ afficher()

Bonnes pratiques

Déclarer chaque classe dans un fichier `NomDeLaClasse.hpp` ou `.h`

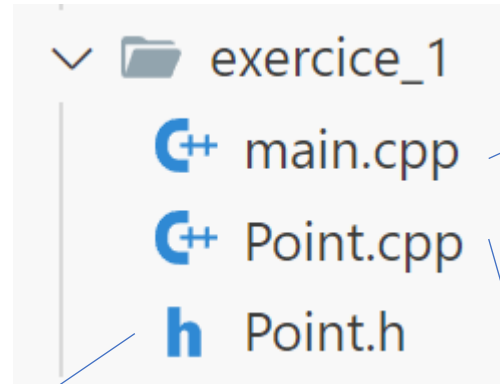
Définir les méthodes dans un fichier `NomDeLaClasse.cpp`

```
// Utilisation
```

```
int main() {  
    Point p1, p2;  
    p1.x = 5;  
    p1.y = 2;  
    p1.lettre = 'A';  
    p1.afficher();           // A (5,2)  
    p2.x = 3;  
    p2.y = -6;  
    p2.lettre = 'B';  
    p2.afficher();           // B (3,-6)  
    p1.afficher();           // A (5,2)  
    return 0;  
}
```

Bonnes pratiques

Architecture d'un projet/exercice :



```
1  #include "Point.h"
2
3  // Utilisation
4  int main() {
5      Point p1, p2;
6      p1.x = 5;
7      p1.y = 2;
8      p1.lettre = 'A';
9      p1.afficher();
10     p2.x = 3;
11     p2.y = -6;
12     p2.lettre = 'B';
13     p2.afficher();
14     p1.afficher();
15     return 0;
16 }
```

```
1  // Déclaration de la classe Point
2
3  #ifndef Point_H
4  #define Point_H
5
6  class Point {
7      public :
8          int x;
9          int y;
10         char lettre;
11         void afficher();
12     };
13
14     #endif
```

```
1  #include "iostream"
2  #include "Point.h"
3
4  // Déclaration en-dehors de la classe
5  void Point::afficher() {
6      std::cout << lettre << "(" << x << "," << y << ")\n";
7  }
```

Bonnes pratiques

Compilation : `g++ *.cpp -o exo1.exe`

Spécificateurs d'accès

- Permet de régler la visibilité des attributs et des méthodes.
- Il existe 3 spécificateurs d'accès :
 - **public (+)** / public : le membre est visible par tous les objets
 - **privé (-)** / private : le membre n'est visible qu'à l'intérieur de la classe
 - **protégé (#)** / protected : notion abordée plus tard (cf. Héritage)

Exemple

- L'attribut **couleur** est public (+) :
 - On peut y accéder et le modifier depuis l'intérieur et l'extérieur (le main() par exemple) de la classe.
- L'attribut **x** est privé (-) :
 - On ne peut ni y accéder ni le modifier depuis l'extérieur de la classe (il est invisible).
 - On peut le modifier depuis les méthodes initialiser(), déplacer(), comparer() et estHorsLimite() car elles sont à l'intérieur de la classe.

Point
- x : int - y : int + couleur : string
+ initialiser(int, int) + déplacer(int, int) + comparer(Point) : bool - estHorsLimite() : bool

Exemple

- L'attribut **x** est privé (-) :
 - La méthode comparer() permet de vérifier si l'objet Point passé en paramètre est le même que l'objet courant (même x et y ou pas).

Les deux objets partageant la classe Point, dans cette méthode, **le x de l'objet passé en paramètre sera visible**, on pourra comparer les attributs x des deux objets Point.

Point
- x : int - y : int + couleur : string
+ initialiser(int, int) + deplacer(int, int) + comparer(Point) : bool - estHorsLimite() : bool

Exemple

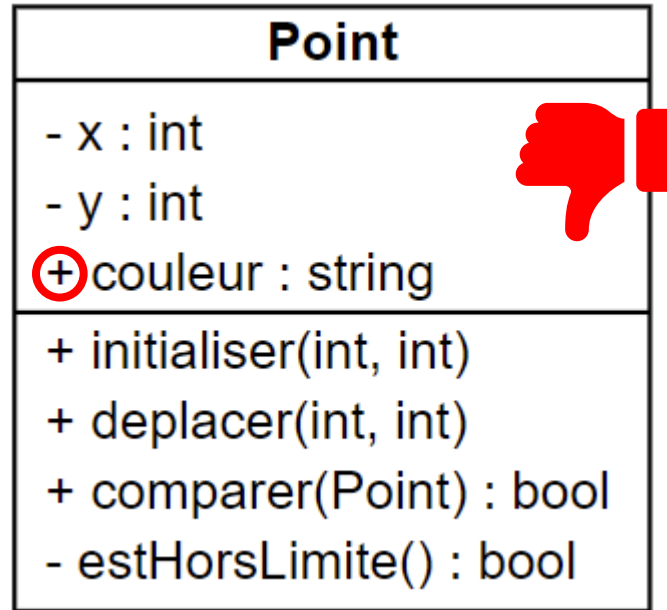
- La méthode **initialiser()** est publique (+) :
 - Elle peut être appelée depuis l'intérieur et l'extérieur (le main() par exemple) de la classe.
- La méthode **estHorsLimite()** est privée (-) :
 - On ne peut pas l'appeler depuis l'extérieur de la classe (elle est invisible)
 - On peut l'appeler depuis les méthodes initialiser(), déplacer(), comparer() et estHorsLimite() (récursivité) car elles sont à l'intérieur de la classe.

Point
- x : int - y : int + couleur : string
+ initialiser(int, int) + déplacer(int, int) + comparer(Point) : bool - estHorsLimite() : bool

Encapsulation

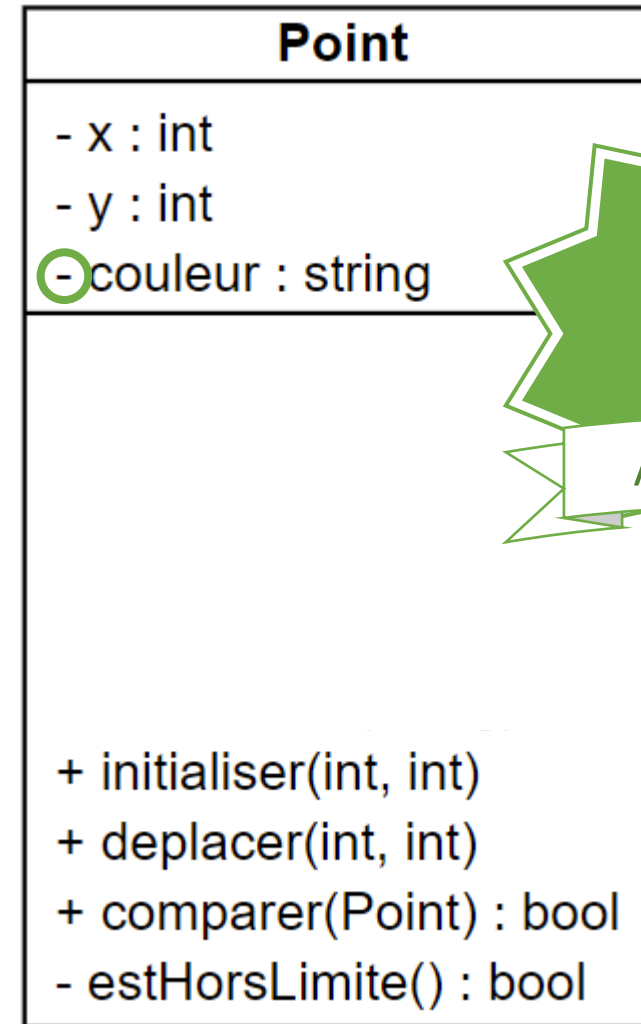
- L'encapsulation consiste à **cacher l'état interne d'un objet** et d'imposer de passer par des méthodes permettant un **accès sécurisé à l'état de l'objet** (contrôle sur les données).
- Comment la mettre en œuvre :
 1. Privatiser l'accès aux attributs
 2. Créer les méthodes d'accès (nécessaires) aux attributs
 - En lecture (**get** / accesseur)
 - En écriture (**set** / mutateur)

Exemple



Accesseur ?

Mutateur ?



Constructeur

Méthode permettant d'initialiser les attributs d'un objet **lors de sa déclaration**.

```
class Point {  
    private :  
        int x;  
        int y;  
    public :  
        Point (int, int);    // Constructeur = pas de type retour  
        void deplacer (int, int);  
        void afficher();  
};  
Point::Point (int abs, int ord) {  
    x = abs;  
    y = ord;  
}
```

```
// Utilisation du constructeur
```

```
int main () {  
    Point a(5, 2); ← // Appel du constructeur  
    a.afficher();    // Je suis en (5, 2)  
    a.deplacer(-2, 4);  
    a.afficher();    // Je suis en (3, 6)  
  
    Point b(1, -1); ← // Appel du constructeur  
    b.afficher();    // Je suis en (1, -1)  
  
    Point c; ← // Erreur de compilation  
    c.afficher();  
  
    return 0;  
}
```



// Le constructeur peut être surdéfini :

```
class Point {  
    private :  
        int x;  
        int y;  
    public :  
        Point ();  
        Point (int);  
        Point (int, int);  
};
```

```
Point::Point() {  
    x = y = 0;  
}
```

```
Point::Point(int val) {  
    x = y = val;  
}
```

```
Point::Point(int abs, int ord) {  
    x = abs;  
    y = ord;  
}
```

// Peut aussi s'écrire :

```
Point::Point(int abs, int ord) : x(abs), y(ord) {}
```

```
// Utilisation du constructeur
int main () {
    Point a(5, 2);        // Appel du constructeur Point(int, int)
    a.afficher();         // Je suis en (5, 2)
    a.deplacer(-2, 4);
    a.afficher();         // Je suis en (3, 6)

    Point b(1, -1);       // Appel du constructeur Point(int, int)
    b.afficher();         // Je suis en (1, -1)

    Point c;              ← // Appel du constructeur Point()
    c.afficher();         // Je suis en (0, 0)

    return 0;
}
```

